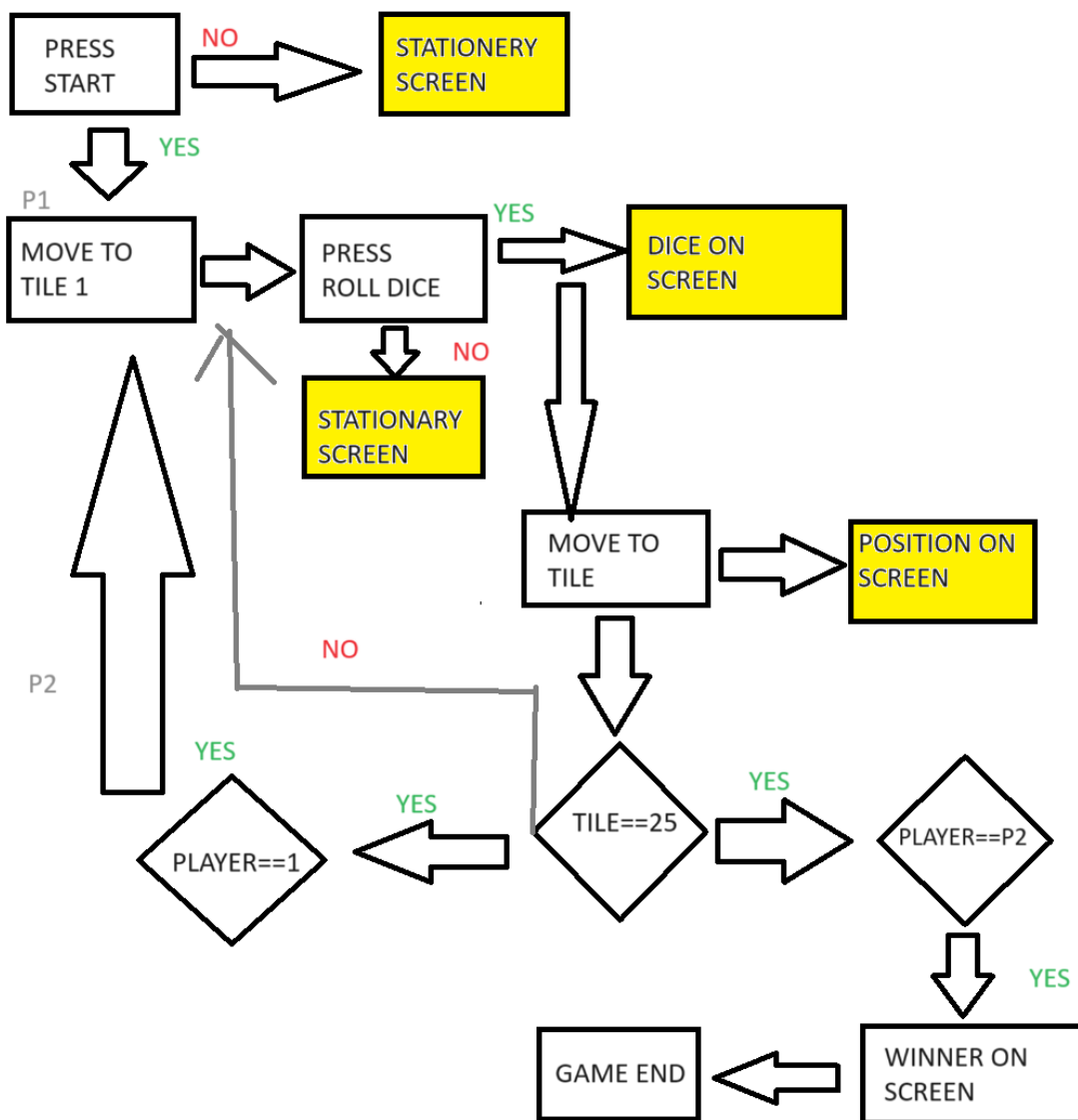


SNAKES AND LADDERS REPORT

(Milestone 3 Report on pg 11)

MILESTONE 2 REPORT

USER FLOW DIAGRAM:



INPUT BLOCK:

- Keeping track of the player: The player will be updated whenever there is someone standing on the first tile.
- Dice and score: The dice will roll each time a player presses a specified “Roll Dice” button on the FPGA. The score will be increased by 1 each time a player rolls the dice and a track of scores of each player will be kept.
- Keeping track of the tile number: The tile number on which the player is standing will be determined by checking which row and column is high at that instant. Then using a matrix configuration, the tile number will be determined.

Inputs	Type	Pin Configuration
clk	In-built clock (100MHz)	W5
r1	PMOD	J1
r2	PMOD	L2
r3	PMOD	J2
r4	PMOD	G2
r5	PMOD	H1
c1	PMOD	A14
c2	PMOD	A16
c3	PMOD	B15
c4	PMOD	B16
c5	PMOD	A17
Start_game	Push button	U18
reset	Push button	W19
Dice_score_trigger	Push button	T17

Tile number:

```
module tile_number(  
    input clk,  
        input r1,  
        input r2,  
        input r3,  
        input r4,  
        input r5,  
        input c1,  
        input c2,  
        input c3,  
        input c4,  
        input c5,  
    output [4:0]tile,  
    output [3:0] tile_lsb,  
    output [3:0] tile_msb  
);  
  
    reg [4:0] tile;  
    reg [3:0] tile_lsb,tile_msb;
```



```
always @(posedge clk)
begin
if (r1 && c1) begin tile<= 5'b00001; tile_lsb<= 4'b0001; tile_msb<=4'b0000; end //1
else if (r1 && c2) begin tile<= 5'b00010; tile_lsb<= 4'b0010; tile_msb<=4'b0000; end //2
else if (r1 && c3) begin tile= 5'b00011; tile_lsb= 4'b0011; tile_msb=4'b0000; end //3
else if (r1 && c4) begin tile= 5'b00100; tile_lsb= 4'b0100; tile_msb=4'b0000; end //4
else if (r1 && c5)begin tile= 5'b00101; tile_lsb= 4'b0101; tile_msb=4'b0000; end //5

else if (r2 && c5) begin tile= 5'b00110 ; tile_lsb= 4'b0110; tile_msb=4'b0000; end //6
else if (r2 && c4) begin tile= 5'b00111; tile_lsb= 4'b0111; tile_msb=4'b0000; end //7
else if (r2 && c3) begin tile= 5'b01000;tile_lsb= 4'b1000; tile_msb=4'b0000; end //8
else if (r2 && c2) begin tile= 5'b01001;tile_lsb= 4'b1001; tile_msb=4'b0000; end //9
else if (r2 && c1)begin tile= 5'b01010; tile_lsb= 4'b0000; tile_msb=4'b0001; end //10

else if (r3 && c1) begin tile= 5'b01011; tile_lsb= 4'b0001; tile_msb=4'b0001; end //11
else if (r3 && c2) begin tile= 5'b01100;tile_lsb= 4'b0010; tile_msb=4'b0001; end //12
else if (r3 && c3) begin tile= 5'b01101;tile_lsb= 4'b0011; tile_msb=4'b0001; end //13
else if (r3 && c4) begin tile= 5'b01110;tile_lsb= 4'b0100; tile_msb=4'b0001; end //14
else if (r3 && c5) begin tile= 5'b01111;tile_lsb= 4'b0101; tile_msb=4'b0001; end //15

else if (r4 && c5) begin tile= 5'b10000;tile_lsb= 4'b0110; tile_msb=4'b0001; end //16
else if (r4 && c4) begin tile= 5'b10001;tile_lsb= 4'b0111; tile_msb=4'b0001; end //17
else if (r4 && c3) begin tile= 5'b10010;tile_lsb= 4'b1000; tile_msb=4'b0001; end //18
else if (r4 && c2) begin tile= 5'b10011;tile_lsb= 4'b1001; tile_msb=4'b0001; end //19
else if (r4 && c1) begin tile= 5'b10100;tile_lsb= 4'b0000; tile_msb=4'b0010; end //20

else if (r5 && c1) begin tile= 5'b10101;tile_lsb= 4'b0001; tile_msb=4'b0010; end //21
else if (r5 && c2) begin tile= 5'b10110;tile_lsb=4'b0010; tile_msb=4'b0010; end //22
else if (r5 && c3) begin tile= 5'b10111;tile_lsb= 4'b0011; tile_msb=4'b0010; end //23
else if (r5 && c4) begin tile= 5'b11000;tile_lsb= 4'b0100; tile_msb=4'b0010; end //24
else if (r5 && c5) begin tile= 5'b11001;tile_lsb= 4'b0101; tile_msb=4'b0010; end //25
else begin tile<= 5'b00000; tile_lsb<= 4'b0000; tile_msb<=4'b0000; end //1
end
endmodule
```

Score counter:

```

1 module score_counter(
2     input clk, trigger, rst,
3     output [3:0] anode,
4     output [7:0] cathode);
5
6     reg[3:0] score_lsb=0;
7     reg[3:0] score_msb=0;
8
9     wire trigger_d;
10
11     reg enable = 1; //enable whenever the button is released
12     debouncer d1(clk, trigger, trigger_d);
13
14     always@(posedge clk)
15     begin
16         if(rst)
17             begin
18                 score_lsb = 0;
19                 score_msb=0;
20                 enable=1;
21             end
22     end

```

```

    if(trigger_d)begin
        if(enable)begin
            score_lsb= score_lsb+1;
            enable=0;
        end
        if (score_lsb>9)begin
            score_lsb=0;
            score_msb = score_msb +1;
            enable=0;
        end
        else if (score_msb >9)
            begin
                score_msb=0;
                score_lsb=0;
                enable=0;
            end
        end
    else
        enable=1;
    end

    wire clk_d;
    clk_div x2(clk, clk_d);
    top_display x1(clk_d, score_lsb, score_msb, anode, cathode);
endmodule

```

Seven segment two digit display:

```

module anode_selector(
    input refresh_clock,
    output reg [3:0] anode=0
);

    always @ (refresh_clock)
    begin
        if (refresh_clock==0)
            anode= 4'b1110;
        else
            anode = 4'b1101;
        end
    endmodule

```

```

module digit_control(
    input [3:0] digit1,
    input [3:0] digit2,
    input refresh_clock,
    output reg [3:0] d_digit=0
);

    always @ (refresh_clock)
    begin
        case (refresh_clock)
            2'd0: d_digit= digit1;
            2'd1: d_digit= digit2;
        endcase
    end
endmodule

```

```
module bcd_to_display(  
    input [3:0] digit,  
    output reg [7:0] cathode=0);  
  
    always @ (digit)  
    begin  
        case(digit)  
            4'd0: cathode= 8'b11000000;  
            4'd1: cathode= 8'b11111001;  
            4'd2: cathode= 8'b10100100;  
            4'd3: cathode= 8'b10110000;  
            4'd4: cathode= 8'b10011001;  
            4'd5: cathode= 8'b10010010;  
            4'd6: cathode= 8'b10000010;  
            4'd7: cathode= 8'b11111000;  
            4'd8: cathode= 8'b10000000;  
            4'd9: cathode= 8'b10010000;  
        endcase  
    end  
endmodule
```



```

module top_Test(
    input clk,input r1,input r2,input r3,input r4,input r5,
    input c1,
    input c2,
    input c3,
    input c4,
    input c5,
    output wire [3:0] anode,
    output wire [7:0] cathode
);
    wire clk_d;
    wire clk_d_de;
    wire rd1,rd2,rd3,rd4,rd5,cd1,cd2,cd3,cd4,cd5;
    wire [4:0] tile;
    wire [3:0] lsb;
    wire [3:0] msb;
    clk_div t1(clk, clk_d);
    tile_number t2(clk_d, r1,r2,r3,r4,r5,c1,c2,c3,c4,c5,tile, lsb,msb);
    top_display t3(clk_d, lsb, msb, anode, cathode);
endmodule

```

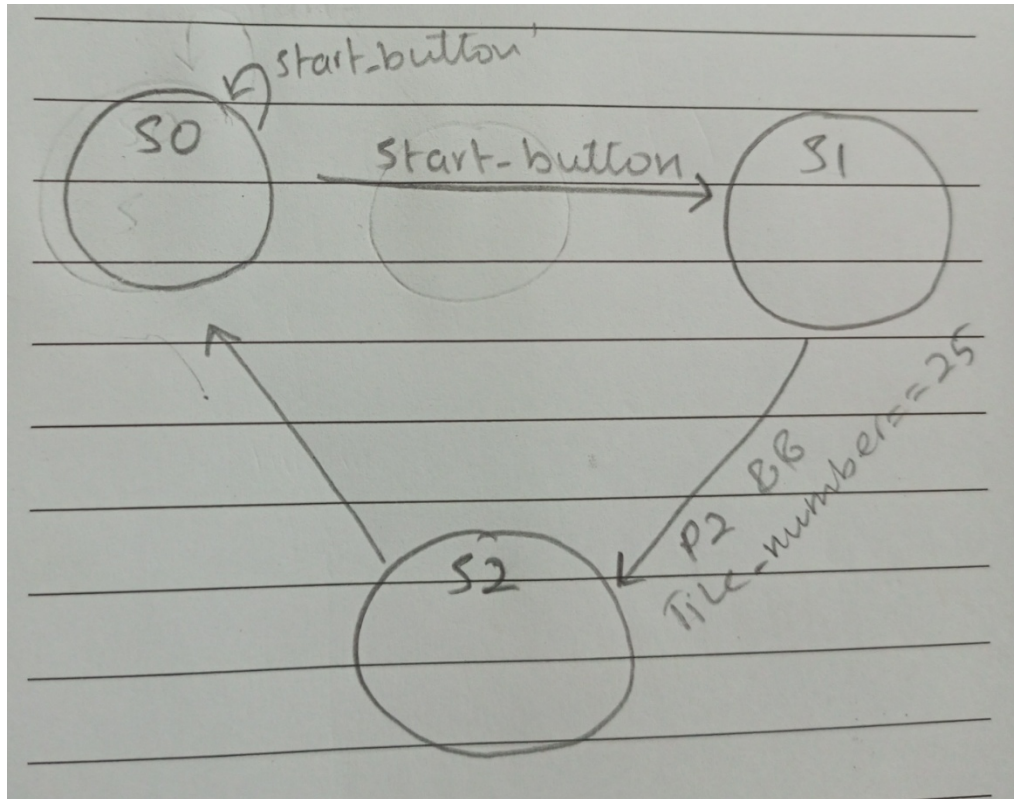
OUTPUT BLOCK:

- Pixel Generator Module (block):
- Generating the grid: Considers the range of a 430 by 480 section of the screen and uses equally divided 25 regions of it th
- e generate a grid. Turns both green and red on when it is within the block to give a yellow color for the tile
- Generating which player is currently active.
- Generating the score of the active player on the seven-segment display.
- VGA SYNC Module (vga_sync):
- Takes the inputs of h_counter and v_counter which return the output of video_on given as an input in the block. It ensures the pixel generator module displays all the pixels whenever h_count and v_count are within the right ranges or whenever the screen is on.
-

CONTROL BLOCK:

STATE	DESCRIPTION	ASSIGNMENT
S0	The board is inactive, and the screen is blank.	2'b00
S1	The board is checking for input and the dice is triggered via the push button. The screen displays the position of the player along with the current dice face. The seven segment would display the score of the active player.	2'b01
S2	The screen displays which player won or	2'b10

	whether it is a tie.	
--	----------------------	--



Milestone 3 Report

State Assignment:

S0 : 00

S1: 01

S2: 10

Truth Table for Next State:

Inputs:

- Reset
- T (Tile player is standing on == 25?)
- P (player number (1 for Player1 and 0 for Player2))
- Q1(t) - current state MSB
- Q0(t) - current state LSB

Outputs:

- Q1(t+1) - Next state MSB
- Q0(t+1) - Next state LSB

Reset=0					
T	P	Q1(t)	Q0(t)	Q1(t+1)	Q0(t+1)
0	0	0	0	0	1
0	0	0	1	1	0
0	0	1	0	1	0
0	0	1 (invalid)	1 (invalid)	X	X
0	1	0	0	0	1
0	1	0	1	1	0
0	1	1	0	1	0
0	1	1 (invalid)	1 (invalid)	X	X
1	0	0	0	0	1
1	0	0	1	1	0
1	0	1	0	1	0
1	0	1 (invalid)	1 (invalid)	X	X
1	1	0	0	0	1
1	1	0	1	1	0
1	1	1	0	1	0
1	1	1 (invalid)	1 (invalid)	X	X

Reset=1					
T	P	Q1(t)	Q0(t)	Q1(t+1)	Q0(t+1)

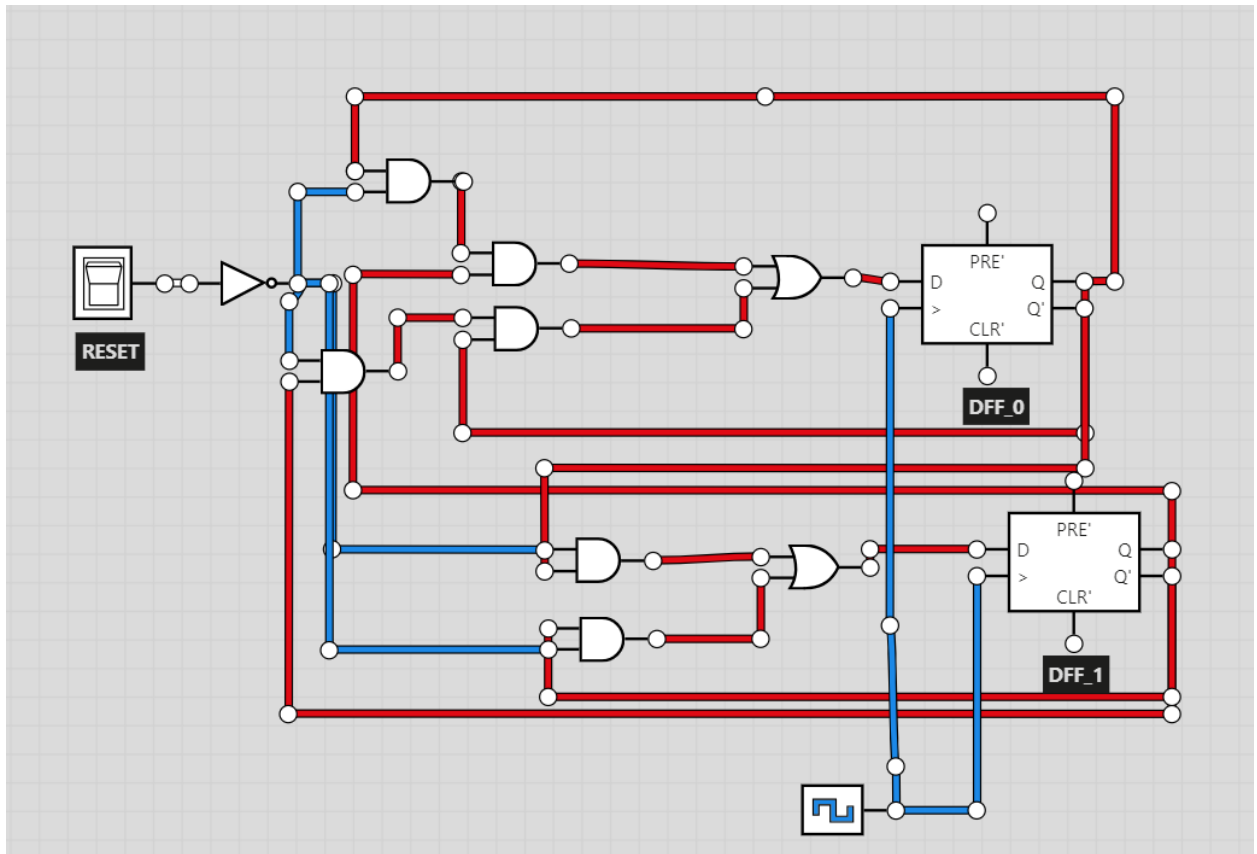
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1 (invalid)	1 (invalid)	X	X
0	1	0	0	0	0
0	1	0	1	0	0
0	1	1	0	0	0
0	1	1 (invalid)	1 (invalid)	X	X
1	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1 (invalid)	1 (invalid)	X	X
1	1	0	0	0	0
1	1	0	1	0	0
1	1	1	0	0	0
1	1	1 (invalid)	1 (invalid)	X	X

Boolean Expressions:

$$Q_0(t+1) = R'Q_0Q_1 + R'Q_0'Q_1'$$

$$Q_1(t+1) = R'Q_0 + R'Q_1$$

Logic Diagram:



State Diagram:

